

NAME: UMUTONIWASE Sonia

RegNo:20251IMA013

PSQL Assignment1

A1 (3 pts). List the first name, last name, and email of all students with a GPA of 3.5 or higher, ordered from highest to lowest GPA.

→**select FIRST_NAME, LAST_NAME, EMAIL, GPA FROM STUDENTS WHERE GPA ≥ 3.5 ORDER BY GPA;**

//I use **SELECT** because it's used to catch and displayed everything from table while GPA was used as condition and order by was used organize the GPA number. //

A2 (3 pts). List all courses in the "Computer Science" department that carry 4 credits, showing course name, course code, and instructor

→ **SELECT course_name, course_code, instructor**

FROM courses

WHERE department = 'Computer Science'

AND credits = 4;

//I use **SELECT** to display all courses in the "computer science" and with department condition. //

A3 (4 pts). Using CASE, produce a report of every student's name and a new column standing that shows 'Honors' for GPA ≥ 3.7, 'Good' for GPA between 3.0 and 3.69, and 'Probation' for anything below 3.0.

SELECT first_name || ' ' || last_name AS student_name,

CASE

WHEN gpa ≥ 3.7 THEN 'Honors'

WHEN gpa BETWEEN 3.0 AND 3.69 THEN 'Good'

ELSE 'Probation'

END AS standing

FROM students;

//I use **CASE** to arrange GPA according to their number from high to low after I displayed their names as full name using select. //

A4 (5 pts). List all students whose email domain is email.com and who enrolled on or after 1-

JAN-2024, showing name and enrollment date, sorted by enrollment date. (Use enrollment_date

from STUDENTS, not ENROLLMENTS.)

SELECT first_name || ' ' || last_name AS student_name,

```
        enrollment_date

FROM students

WHERE email LIKE '%@email.com'

        AND enrollment_date >= DATE '2024-01-01'

ORDER BY enrollment_date;

// I use SELECT to display student name after combining from their first name to the last name with enrollement
date and ORDER BY so that I can arrange date. //
```

Section B: Aggregation and Grouping (20 pts)

B1 (4 pts). For each department, show the department name and the average salary of its employees, rounded to 2 decimal places. Order by average salary descending.

Select

```
SELECT d.department_name,

        ROUND(AVG(e.salary), 2) AS average_salary

FROM departments d

JOIN employees e

        ON d.department_name = e.department

GROUP BY d.department_name

ORDER BY average_salary DESC;
```

// After selecting the dept_name, I also use **AVG** to find the average of employee's salary that will around 2 times and join so that I can combine department table with employees and **GROUP BY** to makes the calculation of every department and **ORDER BY** to arrange their salary from low to high. //

B2 (4 pts). Show each course's course name and how many students are enrolled in it (count of rows in ENROLLMENTS per course_id), including courses with zero enrollments. Order by enrollment count descending.

```
SELECT c.course_name,

        COUNT(e.course_id) AS enrollment_count

FROM courses c

LEFT JOIN enrollments e

        ON c.course_id = e.course_id

GROUP BY c.course_name

ORDER BY enrollment_count DESC;
```

// After selecting it I used **LEFT JOIN** so that the course with no names can appear too and **COUNT** the enrollment. //

B3 (4 pts). Using **HAVING**, list only the courses that have more than 5 enrollments.

```
SELECT c.course_name,  
       COUNT(e.course_id) AS enrollment_count  
FROM courses c  
JOIN enrollments e  
  ON c.course_id = e.course_id  
GROUP BY c.course_name  
HAVING COUNT(e.course_id) > 5;
```

// After selecting I also use **HAVING** that is used to filter the groups and show only courses with more than 5 enrollments. //

B4 (4 pts). For each student, show their full name and their average score across all their

enrollments, but only include students who have at least one enrollment with a non-NULL score.

Round the average to 1 decimal place.

```
SELECT s.first_name || ' ' || s.last_name AS full_name,  
       ROUND(AVG(e.score), 1) AS average_score  
FROM students s  
JOIN enrollments e  
  ON s.student_id = e.student_id  
WHERE e.score IS NOT NULL  
GROUP BY s.first_name, s.last_name;
```

// after selecting I also find **AVG** of student's score with rounding once and **JOIN** student table with enrollments and **GROUP BY** the students name. //

B5 (4 pts). Find the highest, lowest, and average GPA across all students in a single query, each

in its own labeled column.

```
SELECT MAX(gpa) AS highest_gpa,  
       MIN(gpa) AS lowest_gpa,  
       AVG(gpa) AS average_gpa  
FROM students;
```

// I use **SELECT** to fetch the Max GPA and Min GPA and I also find **AVG** of the student's GPA//

Section C: Joins (25 pts)

C1 (5 pts). List each enrollment as: student full name, course name, and grade. For all enrollments, using an inner join across the three relevant tables.

```
SELECT s.first_name || ' ' || s.last_name AS student_name,
```

```
       c.course_name,
```

```
       e.grade
```

```
FROM enrollments e
```

```
INNER JOIN students s
```

```
      ON e.student_id = s.student_id
```

```
INNER JOIN courses c
```

```
      ON e.course_id = c.course_id
```

// After selecting I also used **INNER JOIN** to combine 3 tables (student, course and enrollments) through their ID so that I can show their names, course and their grades together. //

C2 (5 pts). List every course together with the department's budget for the department that offers it (match on courses. department to departments. department_name). Include courses even if no matching department budget is found.

```
SELECT c.course_name,
```

```
       d.budget
```

```
FROM courses c
```

```
LEFT JOIN departments d
```

```
      ON c.department = d.department_name;
```

// After selecting course _name and department budget I also use **LEFT JOIN** to keeps all courses, even when a course has no matching department. //

C3 (5 pts). Using a self-join on EMPLOYEES, list every employee's name next to their manager's name. For employees with no manager, show 'No Manager' instead of NULL.

```
SELECT e.first_name || ' ' || e.last_name AS employee_name,
```

```
       NVL(m.first_name || ' ' || m.last_name, 'No Manager') AS manager_name
```

```
FROM employees e
```

```
LEFT JOIN employees m
```

```
      ON e.manager_id = m.employee_id;
```

// After selecting employees name I also use **NVL** to shows “No Manager” when there is no manager. Instead of null and **SELF JOIN** is used because the manager is also stored in the employees table. //

C4 (5 pts). List all students who have no enrollments at all (use a JOIN and filter for the missing side; do not use a subquery here).

```
SELECT s.first_name || ' ' || s.last_name AS student_name
```

```
FROM students s
```

```
LEFT JOIN enrollments e
```

```
    ON s.student_id = e.student_id
```

```
WHERE e.student_id IS NULL;
```

// After selecting student name I also use **LEFT JOIN** finds students with no matching enrollment, and **IS NULL** identifies them. //

C5 (5pts). List every department along with the name of its manager (join

DEPARTMENTS.manager_id to EMPLOYEES.employee_id), and the number of employees in

that department (from EMPLOYEES.department matching by name). One row per department.

```
SELECT d.department_name,
```

```
    NVL(m.first_name || ' ' || m.last_name, 'No Manager') AS manager_name,
```

```
    COUNT(e.employee_id) AS employee_count
```

```
FROM departments d
```

```
LEFT JOIN employees m
```

```
    ON d.manager_id = m.employee_id
```

```
LEFT JOIN employees e
```

```
    ON d.department_name = e.department
```

```
GROUP BY d.department_name,
```

```
    m.first_name,
```

```
    m.last_name
```

```
ORDER BY d.department_name;
```

// After selecting and i also use **COUNT** to count the number of employees and **LEFT JOIN** to keeps every department and connects it to its manager and employees grouped their dept_name with employees' name order by dept_name so that it can be arranged through alphabetic//

Section D: Subqueries (20 pts)

D1 (5 pts). Using a subquery (not a join), list the names of students who are enrolled in "PL/SQL

Programming" (course code CS202).

```
SELECT first_name || ' ' || last_name AS student_name
```

```

FROM students
WHERE student_id IN (
    SELECT student_id
    FROM enrollments
    WHERE course_id = (
        SELECT course_id
        FROM courses
        WHERE course_code = 'CS202'
        AND course_name = 'PL/SQL Programming'
    )
);

```

// After selecting students name I also use **IN** and **subqueries** to find students whose IDs belong to the selected course. //

D2 (5 pts). Using a correlated subquery, list every course whose average enrollment score is above the overall average score across all enrollments.

```

SELECT c.course_name,
       AVG(e.score) AS average_score
FROM courses c
JOIN enrollments e
    ON c.course_id = e.course_id
GROUP BY c.course_name
HAVING AVG(e.score) > (
    SELECT AVG (e2. score)
    FROM enrollments e2
);

```

// After selecting, I also use the **SUBQUERY** to find the overall average score, and the main query compares each course average with it. //

D3 (5 pts). Find the employee(s) with the highest salary in their own department, using a correlated subquery (i.e., the employee's salary equals the max salary among employees in the same department).

```

SELECT e.first_name,

```

```

    e.last_name,
    e.salary,
    e.department
FROM employees e
WHERE e.salary = (
    SELECT MAX(e2.salary)
    FROM employees e2
    WHERE e2.department = e.department
);
// After selecting the name, salary of employees and their department I also use the correlated subqueries to find the highest amount of employee's salary. //

```

D4 (5 pts). Using IN with a subquery, list the names of courses that have never had a single enrollment.

```

SELECT course_name
FROM courses
WHERE course_id NOT IN (
    SELECT course_id
    FROM enrollments
);
// After selecting it, I also use NOT IN to find courses whose IDs are not found in the enrollment table. //

```

Section E: Data Definition & Manipulation (15 pts)

E1 (4 pts). Add a new column advisor_id NUMBER to the STUDENTS table, referencing employees(employee_id). Write the ALTER TABLE statement (do not just describe it).

```

ALTER TABLE students
ADD advisor_id NUMBER;

ALTER TABLE students
ADD CONSTRAINT fk_students_advisor
FOREIGN KEY (advisor_id)
REFERENCES employees(employee_id);
// I used ALTER to add new column and making new relationship to an employee through advisor_id. //

```

E2 (4 pts). Write an UPDATE statement that raises the salary of every employee in the "Engineering" department by 5%.

UPDATE employees

SET salary = salary * 1.05

WHERE department = 'Engineering';

*// I use **UPDATE** to change employee's salary by multiplying it with 1.05 through department condition. //*

E3 (3 pts). Write a DELETE statement that removes all enrollments with status = 'Active' and a grade of 'C' and a score below 73. Before writing the DELETE, write the equivalent SELECT you used to confirm which rows would be affected (include both in your submission).

SELECT *

FROM enrollments

WHERE status = 'Active'

AND grade = 'C'

AND score < 73;

DELETE FROM enrollments

WHERE status = 'Active'

AND grade = 'C'

AND score < 73;

*// After selecting data from enrollment to confirm that it is there I also **DELETE** it from enrollment table. //*

E4 (4 pts). Create a view called honor_roll that shows the full name, GPA, and enrollment date of every student with a GPA of 3.7 or above, ordered by GPA descending. Then write the single SELECT * FROM honor_roll query that would be used to check it.

CREATE VIEW honor_roll AS

SELECT first_name || ' ' || last_name AS full_name,

gpa,

enrollment_date

FROM students

WHERE gpa >= 3.7

ORDER BY gpa DESC;

SELECT *

FROM honor_roll;

// I create **VIEW** to save honor_roll query and **SELECT *** to check the view. //

Section F: Bonus (up to 5 pts extra credit)

F1 (bonus pts). Using a window function (RANK() or DENSE_RANK()), rank students by GPA within their own enrollment cohort (group by enrollment_date, treating each distinct date as a cohort). Show student name, enrollment date, GPA, and rank.

A perfect SQL answer submitted outside the correct branch, or pushed to main, receives 0 for the assignment regardless of query quality.

SELECT first_name || ' ' || last_name AS student_name,

enrollment_date,

gpa,

RANK () OVER (

PARTITION BY enrollment_date

ORDER BY gpa DESC

) AS rank

FROM students;

/ After selecting it I also use **RANK ()** that is going to gives the student a GPA ranking, while **PARTITION BY** that is going to makes the ranking start separately for each enrollment date. //